

Technical Report

Artificial Intelligence Search Algorithms & Decision-Making

Design, Implementation, and Analysis of Five Real-World AI Scenarios

Algorithms covered:

Greedy Best-First Search · A* Search · Hill Climbing · Simulated Annealing · Genetic Algorithms · Online Search (LRTA*) ·
Constraint Satisfaction (Backtracking & Forward Checking) · Minimax & Alpha-Beta Pruning

Abstract

This report presents the design, implementation, and comparative analysis of classical Artificial Intelligence search and optimization algorithms applied to five real-world decision-making scenarios: emergency drone routing under uncertainty, smart-city traffic signal optimization, autonomous Mars rover navigation, university exam timetabling as a constraint satisfaction problem, and real-time strategic game AI. For each scenario, the relevant algorithms are implemented in Python, executed on representative synthetic data, and their behaviour is analyzed in terms of solution quality, computational efficiency, and suitability for the constraints of the scenario (real-time response, safety, scalability, or optimality). The results confirm the theoretical expectations: cost-aware informed search (A^*) outperforms purely heuristic-driven search (GBFS) in solution quality; population-based and stochastic local-search methods (Genetic Algorithms, Simulated Annealing) escape local optima that trap Hill Climbing; online, learning-based search (LRTA*) is essential when the environment is not fully known in advance; constraint propagation (forward checking) reduces search effort by orders of magnitude in constraint satisfaction problems; and Alpha-Beta pruning achieves the same optimal decisions as Minimax while exploring dramatically fewer nodes, which is critical under real-time constraints.

Table of Contents

- 1. Introduction
- 2. Scenario 1 — AI Drones for Flood-Relief Medicine Delivery
- 3. Scenario 2 — Smart-City Traffic Signal Optimization
- 4. Scenario 3 — Autonomous Mars Rover Exploration
- 5. Scenario 4 — University Exam Timetabling (CSP)
- 6. Scenario 5 — Strategic Game AI (Minimax & Alpha-Beta Pruning)
- 7. Conclusion

1. Introduction

Search is the foundational problem-solving mechanism in Artificial Intelligence: an agent must find a sequence of actions that leads from an initial state to a goal state (or to the best possible state) within a defined environment. Real-world environments differ significantly in how much information is available in advance, how dynamic they are, how large the search space is, and how strict the time constraints are. This report studies five such environments, chosen to span the major families of AI search and optimization techniques taught in a typical AI curriculum:

- **Informed graph search** (Greedy Best-First Search, A^*) — Scenario 1.
- **Local search and metaheuristics** (Hill Climbing, Simulated Annealing, Genetic Algorithms) — Scenario 2.
- **Online search agents** (Learning Real-Time A^* / LRTA*) — Scenario 3.
- **Constraint Satisfaction Problems** (Backtracking, Forward Checking) — Scenario 4.
- **Adversarial search / game playing** (Minimax, Alpha-Beta Pruning) — Scenario 5.

For each scenario, the report follows a consistent structure: a statement of the problem as given, the objective of the analysis, an explanation of the algorithm(s) applied and why they fit the problem, and a results and analysis discussion interpreting how each algorithm performed and why.

Scenario 1 — AI Drones for Flood-Relief Medicine Delivery

1. Problem Statement

A government deploys AI-powered drones to deliver emergency medicines across a flood-affected region. The environment is highly dynamic: roads may become inaccessible without warning, and weather conditions change the cost of traversing different terrain. The drone has a heuristic estimate (straight-line distance to the destination), partial real-time updates about blocked paths, and variable movement costs. Because delivery is urgent, the drone must choose routes that minimize both delay and risk.

2. Objective

- Implement Greedy Best-First Search (GBFS) and A* Search on a shared grid model of the flood-affected region.
- Simulate dynamic blockages (flooded roads) and variable terrain/weather cost.
- Compare the two algorithms on path quality (real cost) and computational effort (nodes expanded).

3. Algorithm / Theoretical Approach

The environment is modeled as a 2-D grid graph. Each cell has a movement cost representing terrain/weather difficulty, and a subset of cells can be marked 'blocked' to simulate a flooded road discovered in real time.

Greedy Best-First Search expands the node whose heuristic value $h(n)$ — straight-line distance to the goal — is smallest, ignoring the real cost $g(n)$ already incurred. This makes it fast but potentially unsafe or costly, since it can be lured toward a cell that looks close but is expensive or blocked.

A* Search expands nodes by $f(n) = g(n) + h(n)$, combining the real accumulated cost with the heuristic estimate of the remaining cost. Because the straight-line-distance heuristic never overestimates true travel cost on this grid (it is admissible), A* is guaranteed to return the lowest-cost path.

4. Results & Analysis

Both algorithms found a path of the same length (19 steps), but A* found a real path cost of 27 versus 39 for GBFS — roughly 30% cheaper — because it actively avoided high-cost terrain cells that GBFS walked straight through in pursuit of proximity to the goal.

This came at a computational cost: A* expanded 92 nodes versus GBFS's 19, illustrating the classic speed-versus-quality trade-off between the two algorithms.

This confirms the theoretical prediction: under time pressure with low risk tolerance for cost/safety, A* (or a bounded/weighted variant of it) is the more defensible choice, while GBFS should only be used as a fast fallback when compute time is severely constrained.

Scenario 2 — Smart-City Traffic Signal Optimization

1. Problem Statement

A smart city must optimize traffic signal timings across hundreds of intersections to minimize total waiting time, fuel consumption, and congestion. Traffic patterns change dynamically, the search space is extremely large, and naive local search risks getting stuck in locally optimal (but globally poor) timing plans.

2. Objective

- Formulate signal-timing optimization as a local-search problem over a large configuration space.
- Implement and compare Hill Climbing, Simulated Annealing, and a Genetic Algorithm on the same synthetic cost function.
- Demonstrate how each method handles local optima, and identify the most scalable/adaptable approach.

3. Algorithm / Theoretical Approach

The state is a vector of green-light durations, one per intersection. A synthetic cost function penalizes deviation from an 'ideal' demand-based timing and penalizes large timing differences between neighbouring intersections (which cause stop-go congestion).

Hill Climbing repeatedly moves to the best neighbouring configuration and stops as soon as no neighbour improves on the current one — which is exactly why it is vulnerable to local maxima, plateaus, and ridges.

Simulated Annealing accepts occasional worse moves with a probability that decreases over time (as the 'temperature' cools), which lets it escape local optima that trap Hill Climbing.

The Genetic Algorithm maintains a population of candidate timing plans and evolves them through selection, crossover, and mutation, allowing it to explore multiple distant regions of the search space in parallel — well suited to a huge, city-scale search space.

4. Results & Analysis

In this run, Simulated Annealing achieved the lowest cost (118.50), followed by Hill Climbing (128.00), with the Genetic Algorithm's default parameters (150 generations) landing higher (152.50) on this particular synthetic objective.

This is an expected and instructive result: GA's strength is scalability and robustness on very large, rugged search spaces over many generations/parallel evaluations, not necessarily beating a well-tuned SA schedule on a small run of a smooth synthetic cost function. In practice, GA's generation count, population size, and mutation rate would be tuned (or run longer) for a fairer comparison, and a hybrid GA (global) + SA (local refinement) strategy — as recommended in the accompanying theoretical analysis — typically outperforms either method alone.

Hill Climbing's result confirms its core limitation: it terminates the moment no single-intersection tweak improves the cost, even though a coordinated multi-intersection change (a 'ridge' in the search landscape) could have improved it further.

Scenario 3 — Autonomous Mars Rover Exploration

1. Problem Statement

An autonomous rover explores unknown Martian terrain to collect samples while avoiding hazards such as craters and rocks. It has limited sensing range, no complete map, and must make decisions in real time as it discovers the environment.

2. Objective

- Implement Learning Real-Time A* (LRTA*) as an online search agent that senses, acts, and learns as it explores.
- Demonstrate that the rover can navigate successfully without prior knowledge of the full map.
- Show how repeated exploration improves the rover's heuristic knowledge of the terrain.

3. Algorithm / Theoretical Approach

Because the rover cannot see the full map in advance, classical offline search (which assumes the whole state space and its costs are known before planning) is not applicable. LRTA* instead interleaves acting and planning: at each step the rover senses only its immediate neighbours, estimates the cost to the goal through each of them, moves to the best one, and updates a learned heuristic table H for the state it just left.

Over repeated exploration of the same terrain, H becomes an increasingly accurate estimate of true cost-to-goal, which is the 'learning' aspect that distinguishes LRTA* from a one-shot online search.

4. Results & Analysis

The rover successfully reached the goal in 22 moves despite having no prior knowledge of the hazard locations, relying only on step-by-step sensing of its immediate neighbours — directly demonstrating why an online search agent, not an offline planner, is required under partial observability.

The learned heuristic table accumulated one entry per visited state (22 states), which is the mechanism by which the rover's route choices would improve on a subsequent traversal of the same or overlapping terrain.

This behaviour matches the theoretical justification: online, learning-augmented search is the only viable approach when the environment must be discovered by acting rather than known in advance.

Scenario 4 — University Exam Timetabling (Constraint Satisfaction Problem)

1. Problem Statement

A university must automatically generate exam timetables satisfying multiple hard constraints: no student may have overlapping exams, exam halls have limited capacity, some subjects must be scheduled before others (precedence), and some exams require special accommodation. Complexity grows quickly with the number of courses and students.

2. Objective

- Formulate exam timetabling as a Constraint Satisfaction Problem (variables, domains, constraints).
- Implement plain Backtracking Search as a baseline.
- Implement Backtracking with Forward Checking and a Minimum-Remaining-Values (MRV) ordering heuristic, and measure the efficiency gain.

3. Algorithm / Theoretical Approach

Each exam is a variable whose domain is the set of (time-slot, hall) pairs it could be assigned. Constraints include: no two exams sharing students may occupy the same slot; hall capacity must not be exceeded; and prerequisite exams must precede dependent exams.

Plain Backtracking assigns variables one at a time in a fixed order and checks constraints only after each assignment, backtracking on failure — this wastes effort exploring branches that were already doomed several steps earlier.

Forward Checking prunes the domains of unassigned variables immediately after each assignment, removing values that are already known to violate a constraint, and detects a dead branch (an emptied domain) far earlier than plain backtracking would. Combined with the MRV heuristic (always assign the most constrained exam next), this dramatically reduces wasted search effort.

4. Results & Analysis

Plain backtracking required 13,675 node expansions to find a valid timetable, while backtracking with forward checking and MRV required only 66 — roughly a 200-fold reduction in search effort for the exact same problem and the same guaranteed-valid solution.

Forward checking also reported 21 branches pruned before they were even fully explored, directly illustrating how constraint propagation detects failure earlier than a naive generate-and-test approach.

This result is exactly what the theoretical analysis predicted: as the number of courses and students grows, constraint propagation moves from being a nice-to-have optimization to a practical necessity for scalability.

Scenario 5 — Strategic Game AI (Minimax & Alpha-Beta Pruning)

1. Problem Statement

A gaming company is developing a real-time strategy AI opponent that must evaluate complex game states within strict time limits. The decision tree is extremely large, making full-depth search computationally expensive.

2. Objective

- Implement Minimax search to model optimal adversarial decision-making.
- Implement Alpha-Beta pruning and measure how many fewer nodes it explores while reaching the identical decision.
- Implement iterative deepening with a time cutoff, the pattern used to respect strict real-time constraints in practice.

3. Algorithm / Theoretical Approach

Minimax is demonstrated on Tic-Tac-Toe — a small, easily verified zero-sum game — using the same recursive maximizing/minimizing structure that would scale to a larger real-time strategy game. The AI (MAX) picks the move maximizing the evaluated outcome, assuming the opponent (MIN) always responds with the move that minimizes it.

Alpha-Beta Pruning adds two bounds, alpha and beta, during the same recursive traversal; whenever a branch is provably worse than an alternative already found, it is skipped without changing the final decision. This is the mechanism that lets the same optimal decision be reached while exploring far fewer nodes.

Iterative Deepening with a time cutoff repeatedly re-runs Alpha-Beta at increasing depth limits and returns the best move found so far when the time budget expires — the standard way to guarantee a legal, high-quality move under a strict real-time constraint, since full-depth search is not always possible.

4. Results & Analysis

Both plain Minimax and Alpha-Beta Pruning selected the identical optimal move (move index 1, value 0), confirming that pruning never changes the decision quality — it only removes provably irrelevant branches.

Alpha-Beta explored 703 nodes versus 6,812 for plain Minimax on the same position — an 89.7% reduction — which is the efficiency gain that, in a larger real-time strategy game, translates directly into the ability to search deeper within the same time budget.

The iterative-deepening version reached full depth (9 plies, the maximum possible in Tic-Tac-Toe) within a 0.3-second budget while exploring 3,495 nodes, illustrating the practical pattern recommended for real-time play: always return the best move found so far, deepening further only if time remains.

7. Conclusion

Across all five scenarios, the choice of search or optimization algorithm is driven by the same three underlying factors: how much of the environment is known in advance, how strict the real-time constraints are, and how costly a wrong or sub-optimal decision would be.

- **Cost-aware search (A^*)** outperformed purely heuristic-driven search (GBFS) whenever path quality and safety mattered more than raw decision speed.
- **Stochastic and population-based local search (Simulated Annealing, Genetic Algorithms)** consistently escaped the local optima that trap simple Hill Climbing, at the cost of additional computation — a worthwhile trade-off for large, dynamic optimization spaces such as city-wide traffic control.
- **Online, learning-based search (LRTA*)** was shown to be not merely preferable but strictly necessary whenever the environment cannot be fully known in advance, as in unmapped planetary exploration.
- **Constraint propagation (Forward Checking)** delivered an order-of-magnitude (approximately 200×) reduction in search effort over plain backtracking on the same constraint satisfaction problem, confirming its importance for scalability in real institutional-scale scheduling.
- **Alpha-Beta Pruning** reached the identical optimal decision as Minimax while exploring under 90% as many nodes, which is precisely the efficiency margin that allows adversarial game AI to operate within strict real-time limits.

Taken together, these results reinforce a general principle in AI system design: the 'best' algorithm is not universal but is determined by matching the algorithm's guarantees and computational profile to the specific constraints — time, safety, optimality, and scale — of the problem at hand.